



자율주행을 위한 객체 탐지 모델 비교

프로젝트 개요

배경

팀원 모두 자율주행 기술에 대한 깊은 관심을 바탕으로, 기존 2D 탐지의 한계를 넘어 실제 주행에 필수적인 3D 공간 인지(Spatial Perception) 기술을 습득하는 것을 목표로 프로젝트를 시작했습니다. 이를 위해 단일 센서의 약점을 상호 보완할 수 있는 Camera, LiDAR, Radar 등 이종 센서 융합(Sensor Fusion) 방식을 채택했습니다. 데이터 전처리부터 모델 학습까지 모든 과정을 진행하면서 고도화된 자율주행 인지 시스템의 핵심 메커니즘을 깊이 있게 이해하고자 이 주제를 선택하게 되었습니다.

목표 및 기대효과

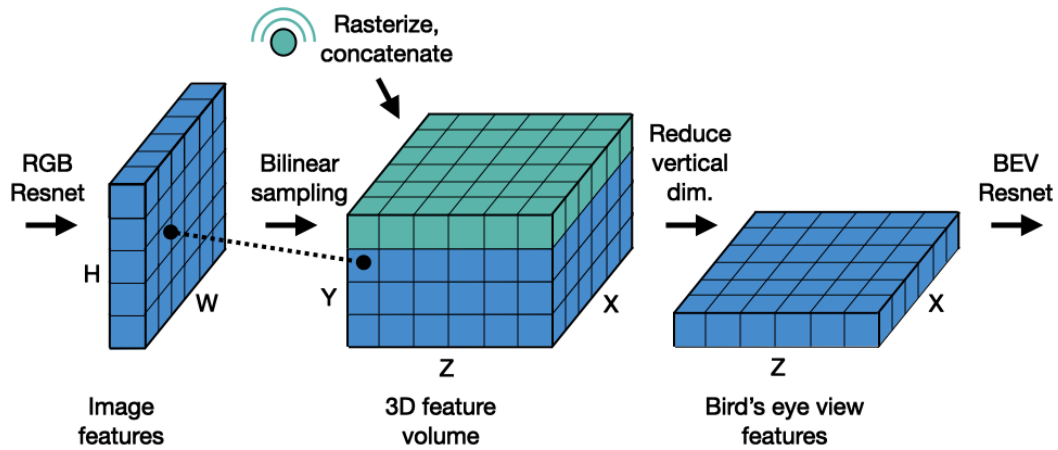
- **목표:** 자율주행 인지의 핵심인 BEV(Bird's Eye View) 상에서 '카메라+라이다'와 '카메라+레이더' 퓨전 방식에 따른 세그멘테이션 성능 차이를 정량적으로 분석하고 실효성을 검증
- **기대효과:** 주행 환경과 하드웨어 비용 제약 조건에 최적화된 하이브리드 센서 퓨전 전략의 근거를 제시하며, 고가 라이다 시스템의 대체 가능성 및 보완 방향에 대한 기술적 가이드라인을 확보

사용 모델 및 데이터

- **Dataset:** [nuScenes Dataset](#) (v1.0-mini)
 - 데이터 선정 이유 : Camera, LiDAR, Radar 데이터를 모두 포함한 멀티모달 데이터셋
 - 데이터 구성 : 6개의 카메라 뷰 + 360도 LiDAR/Radar point cloud
- **Model Architecture:** SimpleBEV (Backbone- ResNet-101)
- **Environment:** Google Colab Pro+

핵심 모델 구조

simpleBEV



Pipeline

1. 이미지 특징 추출(2D)
 - a. 카메라로 들어온 6장의 원본 이미지에서 중요한 특징을 뽑아냄
 - b. feature map 생성
2. 2D → 3D 공간으로 변환
 - a. 납작한 이미지 데이터를 우리가 미리 만들어둔 3D 빈 공간의 정확한 위치에 채워넣음
 - b. Bilinear Sampling : 픽셀과 픽셀 사이의 빈 공간의 값을 추정
3. Voxel Concatenation / 센서 결합
 - a. 3D로 변환된 카메라 데이터와 원래 3D인 LiDAR 또는 Radar 데이터를 하나로 합침
4. BEV decoding
 - a. BEV map 생성 : 높이 정보를 압축(2D BEV Map) → 위에서 내려다본 2D 지도 생성

단계	프로세스	주요 내용
1단계	2D Feature Extraction	각 카메라(Multi-view) 이미지에서 CNN을 통해 시각적 특징 추출
2단계	Lifting (2D to 3D)	추출된 2D 특징을 가상의 3D 그리드(Voxel) 공간으로 투영
3단계	Radar/Lidar Fusion	레이더/라이다 데이터를 동일한 3D 그리드에 매핑하여 카메라의 불확실한 깊이 (Depth) 보완
4단계	3D Aggregation	3D 공간 상에 배치된 카메라와 레이더 특징 데이터를 채널 방향으로 결합
5단계	BEV Projection	3D 데이터를 수직 방향으로 압축(Pooling)하여 2D 평면(Bird's Eye View) 맵 생성
6단계	Prediction Head	생성된 BEV 맵을 입력받아 최종적으로 객체 탐지 및 세그멘테이션 수행

학습 과정 및 시행착오

1. 프로젝트 수행 배경 및 기술 스택 전환

- 초기 목표

- MMdec 기반의 **FCOS3D**를 활용하여 3D Bounding Box를 생성, 자율주행 객체 인지 성능 확보

- 문제 발생

- Google Colab에서의 MMdec 라이브러리 설치 오류 및 빌드 이슈로 인해 초기 파이프라인 구축 실패

- 대안 모색 및 역할 분담

- 환경적 제약을 극복하고 팀원별 강점을 살리기 위해 역할 분담 및 아키텍처 변경

단계	시도한 기술 (Model)	주요 내용 및 결과	변경 사유 (Lesson Learned)
Phase 1	FCOS3D (MMdec)	MMdetection 및 환경 구축	Colab 환경 내 종속성 충돌 및 빌드 실패로 파이프라인 구축 불가
Phase 2	YOLO v8 (2D)	약천후 데이터(ACDC) 학습	데이터 부족 및 단순 2D 인식으로는 자율주행 공간 정보 부족
Phase 3	YOLO v8 (3D)	KITTI 데이터 활용 3D 확장	깊이(Depth) 추론 없이 2D 박스를 늘리는 방식의 기술적 한계 노출
Phase 4	SimpleBEV	Cam+LiDAR / Cam+Radar	프로젝트의 핵심 아키텍처(Main Backbone)로 발전

구분	팀원	센서 구성	백본 (Encoder)	핵심 변수 (Variable)	전략
LiDAR 팀	팀원 1	Cam + Lidar	ResNet-101	<code>lr=5e-4 , res_scale=2</code>	고해상도 베이스라인 구축
	팀원 2	Cam + Lidar	ResNet-101	<code>lr=3e-4 , res_scale=2</code>	가벼운 백본과 LR 변화 관찰
Radar 팀	팀원 3	Cam + Radar	ResNet-101	<code>use_metaradar=True</code>	메타레이더 정보 활용 성능 확인
	팀원 4	Cam + Radar	ResNet-101	<code>nsweeps=6</code>	Radar의 희소성을 Temporal(6 sweeps)로 보완

2. 데이터셋 확보 및 규모 최적화의 시행착오

- 초기 단계

- nuScenes mini 데이터셋으로 베이스라인 모델 구축
- 결과는 유효했으나 저해상도 및 샘플 부족으로 인한 성능 고도화 한계 직면

- 데이터 오해 및 분석

- SDK 활용 미숙으로 초기 train/val 데이터가 중복되었다는 오판
- 성능 정체 원인 파악을 위해 trainval 원본 데이터셋의 일부 활용 결정
- 원본 데이터의 방대한 용량으로 인해 전체 데이터의 약 1/10 선별

- 데이터 무결성 훼손

- 단순히 용량을 줄이기 위해 특정 파일(n008 등)을 무작위 삭제 → Scene 단위의 연속성이 파괴되어 학습 불가능



데이터를 일일이 복구하고 선별하는 데 시간을 쏟기보다는, 차라리 다시 mini 데이터셋으로 돌아가서 실험을 효율적으로 진행하는 게 낫다고 판단

3. 학습 및 성능 개선 방안

camera + LiDAR

- 데이터 증강(Augmentation) 도입

- Mini 데이터셋의 부족한 데이터양 보완하기 위해 Random Flip, Crop 등 다양한 기법을 적극 적용

실험 결과

항목	Baseline (Standard)	Experiment (Augment)	비고
학습 횟수 (Iters)	2500	1300	증강 모델 학습량 부족 가능성
데이터 증강	적용 안 함	Rand_Flip, Rand_Crop	이미지 대칭 및 크롭 적용
학습률 (LR)	기본값	5e-4 (0.0005)	수동 설정 적용
센서 설정	Camera + Lidar (True)	Camera + Lidar (True)	동일 환경
인코더	ResNet-101	ResNet-101	동일 환경

▼ camera + LiDAR - 증강X

```
!python /content/simple_bev/train_nuscenes.py \  
  --exp_name simplebev_standard_v1 \  
  --max_iters 2500 \  
  --log_freq 100 \  
  --save_freq 500 \  
  --batch_size 1 \  
  --data_dir "/content/drive/MyDrive/Colab Notebooks/Objec_detection_pro/v1.0-mini" \  
  --init_dir "" \  
  --res_scale 2 \  
  --encoder_type res101 \  
  --use_lidar True \  
  --do_rgbcompress True \  
  --device_ids '[0]
```

▼ camera + LiDAR - 증강 O

```
!python train_nuscenes.py \  
  --exp_name simplebev_m1_aug_max_v1 \  
  --device_ids '[0]' \  
  --dset mini \  
  --batch_size 1 \  
  --max_iters 1300 \  
  --val_freq 100 \  
  --save_freq 100 \  
  --lr 5e-4 \  
  --res_scale 2 \  
  --encoder_type res101 \  
  --use_lidar True \  
  --data_dir /content/data/sets/nuscenes/ \  
  --log_dir logs_m1_aug \  

```

```
--ckpt_dir checkpoints_m1 \
--rand_flip True \
--rand_crop True \
--do_rgbcompress True
```

loss 1.3428; iou_t 56.6; iou_v 28.3; BEST 28.3

- **결과:** 증강 기법 적용 후 오히려 모델 성능(IoU)이 하락하는 현상 발생
- **원인 분석**
 - **Over-regularization:** 극소량의 데이터셋(**mini**)에 강한 증강이 가해지면서 모델이 특징점을 제대로 학습하지 못하고 편향(Bias) 발생
 - **데이터 복잡도 불일치:** 현재 모델 파라미터 대비 증강된 데이터의 복잡도가 높아 수렴 속도 저하 및 성능 하락 초래

camera + RADAR

- nsweep 파라미터 증가 도입
 - 과거 프레임의 레이더 데이터를 누적하여 점군(Point Cloud) 밀도를 높이고 노이즈 감소
 - 실험 결과
- ▼ camera + Radar -nsweep 낮춤

```
!python train_nuscenes.py \
--exp_name simplebev_full_run_v1 \
--device_ids '[0]' \
--dset mini \
--batch_size 4 \
--max_iters 2200 \
--save_freq 500 \
--lr 5e-4 \
--data_dir /content/data/sets/nuscenes/ \
--use_lidar False \
--use_radar True \
--use_metaradar True \
--do_shuffle_cams True
```

loss 4.57210; iou_t 76.1; iou_v 31.4

- ▼ camera + Radar -nsweep 증가

```
!python train_nuscenes.py \
--exp_name="radar_mine" \
--max_iters=3000 \
--log_freq=100 \
--dset='mini' \
--nsweeps=6 \
--batch_size=4 \
--grad_acc=5 \
--use_scheduler=True \
--data_dir /content/data/nuscenes \
```

```
--log_dir='logs_nuscenes' \
--ckpt_dir='checkpoints' \
--res_scale=2 \
--ncams=6 \
--encoder_type='res101' \
--do_rgbcompress=True \
--use_lidar=False \
--use_radar=True \
--use_metaradar=True \
--device_ids=[0]
```

loss 0.30501; iou_t 76.5; iou_v 74.5

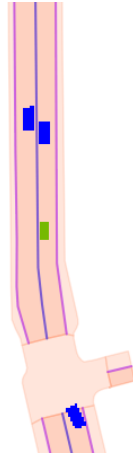
결과: IOU_v 상승 (31.4 → 69.7)

RADAR vs LiDAR

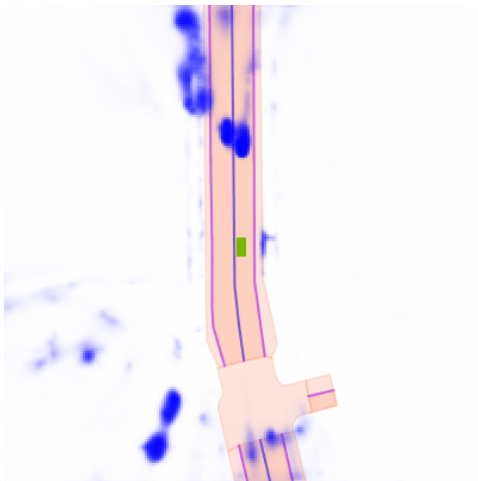
항목	RADAR (Baseline)	Experiment 1	Experiment 2	LiDAR(Baseline)	Experiment 1
학습 횟수 (Iters)	2500	2200	3000 (영상은 2000 에서의 모델을 사용)	2500	1300
데이터 증강	적용 안 함	적용 안 함	적용 안 함	적용 안 함	Rand_Flip, Rand_Crop
학습률 (LR)	기본값	기본값	기본값	기본값	5e-4 (0.0005)
센서 설정	Camera + Radar + Metaradar (True)	Camera + Radar + Metaradar (True)	Camera + Radar + Metaradar (True)	Camera + Lidar (True)	Camera + Lidar (True)
인코더	ResNet-101	ResNet-101	ResNet-101	ResNet-101	ResNet-101
부가정보 활용 (nsweeps)	5	3	6	3	3
IOU_v	60.8	31.4	69.7(74.5에서 꾸준히 상승)	33.3	28.3

최종 성과 및 시각화(Visualization)

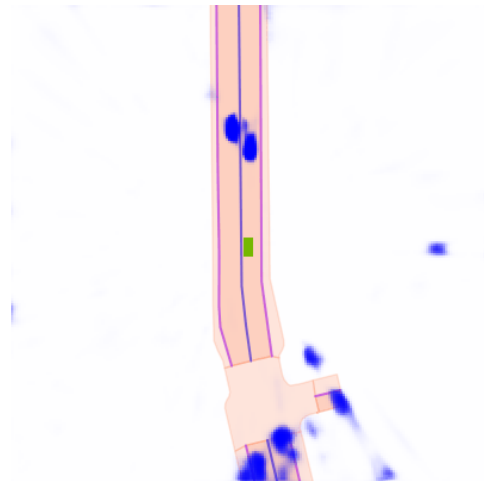
- 결과
 - 학습된 모델을 활용하여 개별 이미지를 연속된 프레임を 엮어 **BEV 상의 객체 인지 시연 영상(Inference Video) 제작함**
- 결론: SimpleBEV 기반의 센서 퓨전(Camera, LiDAR, Radar) 학습 파이프라인 구축 및 동작 검증, 데이터셋 관리의 무결성과 하드웨어 리소스 최적화의 중요성을 체득



gt



LiDAR



RADAR

문제 해결 및 원인 분석

- 데이터: nuScenes mini 데이터셋 기반 학습

camera + LiDAR

- 데이터 증강 적용 기법: Random Flip, Random Crop, Resolution Scaling
 - 모델의 mAP(IoU)가 개선되지 않거나, 오히려 원본 데이터 학습 대비 검증 오차가 증가함

원인 분석

- 기하학적 일관성 결여(Geometric Inconsistency) : SimpleBEV는 2D 이미지를 3D voxel 공간으로 투영할 때 카메라의 내/외부 파라미터(Intrinsics/Extrinsics)를 참조함
 - Random Crop이나 Flip 적용 시 이미지 픽셀의 위치는 변했으나, 이에 대응하는 투영 행렬(Projection Matrix)이 다시 계산되지 않음
 - BEV 평면상에 객체 위치 오정렬(Misalignment)/3D 공간 좌표 왜곡이 발생하여 심각한 학습 노이즈로 작용
- 데이터 절대량 부족 및 모델 복잡도 불일치 : mini 데이터셋(323 샘플)은 ResNet-101과 같은 대규모 파라미터를 가진 인코더가 일반화된 특징을 학습하기엔 물리적으로 부족

- 부족한 데이터셋에 더해진 증강된 이미지는 유의미한 데이터가 아닌 노이즈로 인식되어 과적합(Overfitting)이 발생할 수 있
- **인프라 제약에 따른 실험 범위 제한** : Original 데이터셋(약 400GB+)을 수용할 수 없는 Google Colab의 스토리지 및 VRAM 제약으로 인해 데이터 규모 확장(Scaling up)을 통한 성능 돌파구 마련이 물리적으로 불가능했음

대응 및 향후 과제

- **기하학 기반 증강(Coordinate-aware Augmentation)**
 - 단순 이미지 변환을 넘어 투영 행렬까지 동기화하여 수정하는 증강 로직을 도입
- **데이터 선별 및 큐레이션**
 - 무작위 삭제가 아닌, 객체 밀도가 높은 핵심 Scene을 추출하여 Original 데이터셋의 일부를 활용하는 정교한 데이터 클렌징 전략 수립
- **하이퍼파라미터 최적화**
 - 데이터 규모에 맞춰 증강의 강도를 적절히 낮추는 등, 학습 난이도를 미세하게 조절하여 모델을 안정적 필요

최종 선정

참고 논문 : **Simple-BEV: What Really Matters for Multi-Sensor BEV Perception?**

- **라이다 (빛의 직진성 이용)**
 - **특징**: 빛은 파장이 짧고 직진성이 강해 아주 작은 물체도 맞고 돌아옵니다.
 - **3D 매핑**: 3D 그리드(Voxel) 안에 점을 찍을 때, ****좌표(x, y, z)가 매우 정밀****합니다. 마치 가느다란 펜으로 촘촘하게 점을 찍는 것과 같습니다.
 - **결과**: 카메라 이미지의 픽셀과 라이다의 점을 1:1로 매칭시키기 매우 유리하여, 깊이(Depth) 보완이 실측 수준으로 정확해집니다.
- **레이더 (음파와 유사한 전파 성질 이용)**
 - **특징**: 전파는 파장이 길어 물체에 부딪히면 사방으로 퍼지거나(산란), 물체를 타고 넘어가기도 합니다. 소리가 벽 뒤에서도 들리는 것과 비슷한 원리입니다.
 - **3D 매핑**: 3D 그리드 상에서 점이 **"뭉뚱그려진(Sparse/Noise)"** 상태로 나타납니다. 물체의 정확한 '형태'보다는 '거기에 무언가 움직이는 덩어리가 있다'는 정보에 가깝습니다.
 - **결과**: 카메라의 깊이를 보완할 때, 라이다처럼 정밀한 외곽선을 잡아주기보다는 "이 근처에 물체가 있으니 카메라 특징값들을 이 거리대에 집중시켜라"라는 가이드 역할을 주로 합니다.

- 소스 코드가 포함된 GitHub : <https://github.com/Hyeong-Seon/Object-Detection.git>
- Project 과정을 보여주는 기록을 포함하고 있는 자료 :  [오빠 달려](#)

